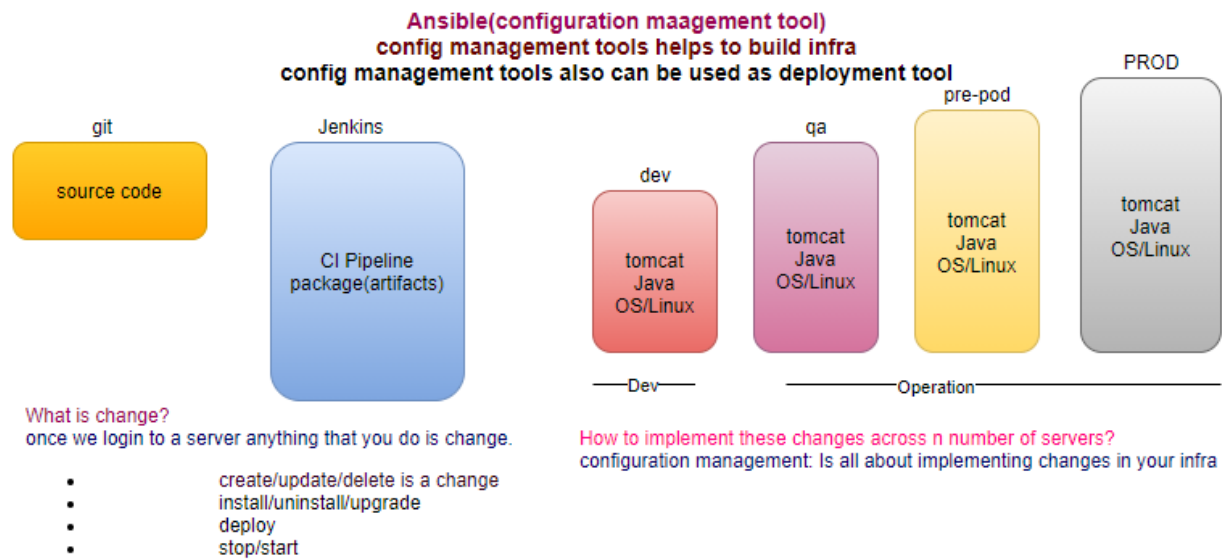


Ansible (Master-Slave agentless Push Model)?



Ansible is a configuration management tool which traditionally **uses push approach** to achieve its objectives, by managing all the servers through one single machine running the Ansible Configuration Management tool.

UC: There are 100 of servers are running in the production environment, your manager comes to you ask to please shutdown all the server/update the path.

You don't do the immediately, first ask for CR, then what will be the impact on the production. When you perform any change in server is called configuration change.

- Why?
- When?
- How? What are the instructions to these changes?
- Who? Will implement
- How? to rollback

CAB (Change Advisory Board): Track all the changes, any change is also called configuration change.

Final goal is to deploy artifacts on the production environment, from where it can be accessed from end users.

Why we need configuration management tool?

UC: we need to install tomcat and java on 1 server? Manually OK

UC: we need to install tomcat and java on 100 servers? Automation/Orchestration

Configuration management **tool login into the** all-slave servers and perform the action on behalf of you.

Login into one system and implement the changes across your infrastructure.

When you join an organization, get Laptop. If any update released in future is DLA comes in your seat or do the updation from their location.

Configuration Management ?

is all about changing something on any system / computer / server

what is change ? any action that is performed on a system/server that alters the state of system/server called a change

such as:

- creating / modifying / deleting file or directories etc.
- install / deploy a new software/application
- upgrading a software
- uninstalling a software

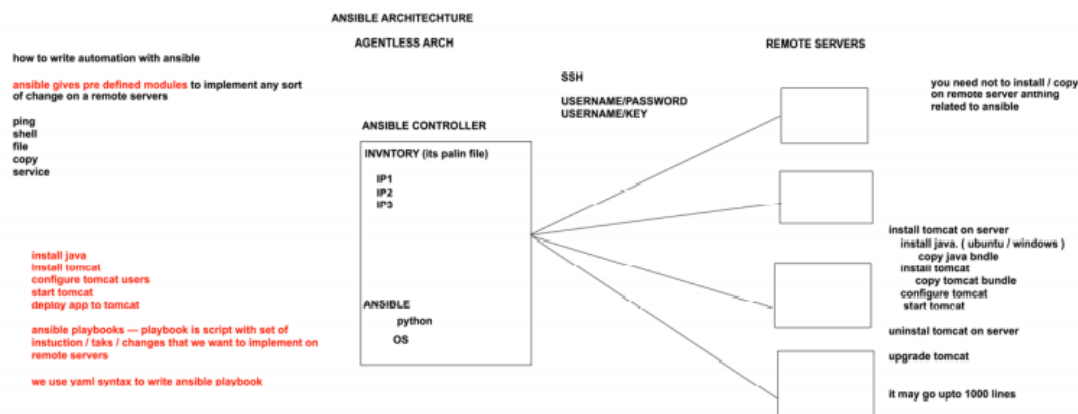
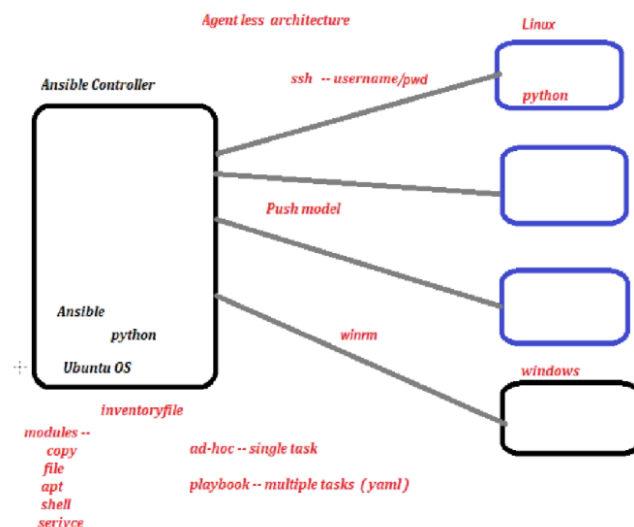
so any change that is done on a system or server is simply called as configuration change

why do we need config management or config management tools ?

it is easy to make the changes in one or two system/server, but in real time we have to do the similar changes across thousands of servers

so it required to automate, implement & maintain the consistency of changes across n number of servers.

hence we use configuration management tools.



step1: Install ansible

ansible --version

/etc/ansible/hosts # inventory

hosts file is inventory file where we define the ip addresses of the target servers.

/etc/ansible/ansible.cfg #configuration

configuration (inventory, sudo-user, host_key_checking=false)

/etc/ansible/roles # roles directory

step2: setup communication with target servers (SSH connection)

L*

How to login into Linux machine using username and password?

by-default, ec2 ubuntu instance have only ubuntu user, your instance doesn't have username and password.

step1: Install ssh client (putty, MobaXterm-portable edition)

*/

step1: create user(devops), in all three instances

adduser devops

step2: set password for this user, in all three instances

passwd devops

step3: switch as devops user

su - devops

step4:

touch file1 # successfully created

apt install httpd -y # denied you need to be root to perform this command

sudo apt install httpd -y

request for password, after enter password, says ansible user is not in sudoers file

step5: This ansible user doesn't have sudo privilege as of now, is you want to give sudo privilege to devops user, switch to root user (only root user can provide).

su -

visudo

root ALL=(ALL) ALL

devops ALL=(ALL) NOPASSWD: ALL

:wq!

step6:

apt install httpd -y # you need to be root to perform this command

sudo apt install httpd -y

#request for password, after enter password, install the httpd

step7: Now establish connection between master and worker

su - devops

ssh <IP> # permission denied (public key)

step8:

exit # as a root user

sudo su -

vi /etc/ssh/sshd_config

permitRootLogin Yes # uncomment

PasswordAuthentication Yes # uncomment

PasswordAuthentication No # comment

service sshd restart

step9:

su - devops

ssh <IP> # Will ask for password every time

step10: Establish Trust relationship Go to ansible server and generate keys.

Trust relationship established at same level (root-> root, ansible>ansible)

su - devops

ssh-keygen

ls -a # o/p .ssh (hidden file)

cd .ssh

ls -l # o/p id-rsa, id-rsa-pub

step11: copy the public key of server to the worker nodes

cd .ssh

ls -l # o/p id-rsa, id-rsa-pub

ssh-copy-id <username>@<IP>

step12: from the controller node ssh worker node

su - ansible

ssh <username>@<IP> # only first time ask for key

cat /etc/ansible/hosts

hostname -i // private ip

hostname -f // dns name

```

# This is the default ansible 'hosts' file.
#
# It should live in /etc/ansible/hosts
#
# - Comments begin with the '#' character
# - Blank lines are ignored
# - Groups of hosts are delimited by [header] elements
# - You can enter hostnames or ip addresses
# - A hostname/ip can be a member of multiple groups
#
# Ex 1: Ungrouped hosts, specify before any group headers.
## green.example.com
## blue.example.com
## 192.168.100.1
## 192.168.100.10
#
# Ex 2: A collection of hosts belonging to the 'webserver' group
## [webserver]
## alpha.example.org
## beta.example.org
## 192.168.1.100
## 192.168.1.110
#
# If you have multiple hosts following a pattern you can specify
# them like this:
## www[001:006].example.com
#
# Ex 3: A collection of database servers in the 'dbserver' group
## [dbserver]
##
## db01.intranet.mydomain.net
## db02.intranet.mydomain.net

```

nano hosts

```

db
10.128.0.11
10.128.0.12

[web]
node1
node2

[app]
node1.us-central1-a.c.devops21-300415.internal
node2.us-central1-a.c.devops21-300415.internal

```

Host pattern

“all” pattern refers to all the machines in an inventory

ansible all --list-hosts

ansible <groupname> --list-hosts

ansible <groupname> [0] --list-hosts

<groupname> [0]: picks the first machine of group

<groupname> [-1]: picks the last machine of group

<groupname> [0:1]: picks the first two machines of group

<groupname> [2:5]: picks the 3,4,5,6 machines of group

group separated by a colon can be used to use hosts from multiple groups

groupname1: groupname2

How controller instruct to the target servers?

- Adhoc commands
- Playbook

What is Module?

Ansible modules are reusable, standalone scripts that can be executed by the Ansible API, or by the ansible or by the ansible-playbook programs.

Modules: everything in Ansible is module

Adhoc commands

How to prepare inventory?

You can modify default inventory file /etc/ansible/hosts

ansible-doc -l

ansible-doc shell

ansible all -m ping -u devops -k (--ask-pass) # Failed Host key checking is enabled

```
root@controller:~# cd /etc/ansible/
root@controller:/etc/ansible# ls -l
total 28
-rw-r--r-- 1 root root 19985 Dec 14 23:49 ansible.cfg
-rw-r--r-- 1 root root 1016 Dec 14 23:49 hosts
drwxr-xr-x 2 root root 4096 Dec 14 23:50 roles
root@controller:/etc/ansible# vi ansible.cfg
root@controller:/etc/ansible# vi hosts
root@controller:/etc/ansible#
root@controller:/etc/ansible# ansible db -m ping -u devops --ask-pass
SSH password:
10.128.0.11 | FAILED! => {
  "msg": "Using a SSH password instead of a key is not possible because Host Key checking is enabled and sshpass does not support this. Please add this host's fingerprint to your known_hosts file to manage this host."
}
10.128.0.12 | FAILED! => {
  "msg": "Using a SSH password instead of a key is not possible because Host Key checking is enabled and sshpass does not support this. Please add this host's fingerprint to your known_hosts file to manage this host."
}
root@controller:/etc/ansible#
```

It just like ssh devops@<IP> and say No

```
root@controller:/etc/ansible# ssh devops@10.128.0.11
The authenticity of host '10.128.0.11 (10.128.0.11)' can't be established.
ECDSA key fingerprint is SHA256:LKJUCHZ7aMmX2D8MTgy0zEaUgNdLP7Cpg0c90mzCax0.
Are you sure you want to continue connecting (yes/no)? no
Host key verification failed.
root@controller:/etc/ansible#
```

How to solve?

nano ansible.cfg

disable the attribute host_key_checking

```
# uncomment this to disable SSH key host checking
host_key_checking = False

# change the default callback, you can only have one 'stdout' type enabled at a time.
#stdout_callback = skippy

## Ansible ships with some plugins that require whitelisting,
## this is done to avoid running all of a type by default.
```

How to create your customize inventory file?

we may have different inventory files for different environments

touch myinv

nano myinv

ansible all -i /home/ansible/myinv -m ping # i (inventory file path)

Adhoc commands with arguments?

#copy module

/*copy module copy from controller to target, source must be on controller machine, and file must be existing on the controller machine else it will not be executed". */

ansible all -i /home/ansible/myinv -m copy -a "src=/tmp/neeraj.txt dest=/tmp"

ansible all -m copy -a "src=/tmp/devops.txt dest=/tmp backup=yes" -u devops -k

file module helps you to create file or directory

ansible all -i /home/ansible/myinv -m file -a "path=/tmp/devops state=directory/touch"

#shell module (which command want to execute on target server)

ansible all -i /home/ansible/myinv -m shell -a "ls -a /tmp" -u devops -k

ansible all -m shell -a "df -h" -u devops -k --limit node1,node2

ansible all -i /home/ansible/myinv -m shell -a "df -h /" -u devops -k // disk utilization


```
ansible all -i /home/ansible/myinv -m shell -a "free -m" -u devops -k // memory utilization
```

```
ansible all -i /home/ansible/myinv -m shell -a "service cron status" -u devops -k
```

setup module

just displays the Gather facts (**ansible collects information about worker nodes before executing any command**)

```
ansible all -i <path of inventory file> -b -m setup
```

#src must be on server and must be available (Idempotence no change)

```
[web]
10.154.0.27 ansible_user=devops ansible_ssh_pass=today@1234
10.154.0.28 ansible_user=devops ansible_ssh_pass=today@1234
```

```
ansible web -i prodenv -m ping # i-> inventory
```

How to limit your execution?

```
ansible dev -i /home/ansible/myinv -m shell -a "df -h" --limit=< IP>
```

How to execute commands as a root user? (run as an ansible user)

```
ansible all -i /home/ansible/myinv -m file -a "path=/opt/devops state=directory"
```

permission denied

by default, Ansible perform any action **as an ansible user**

```
drwxrwxr-x  2 ansible ansible 4096 Aug  1 17:19 devops
drwxr-xr-x 16 root      root   4096 Aug  1 14:55 labs
-rw-rw-r--  1 ansible ansible  0 Aug  1 17:17 naresh.txt
drwx-----  3 root      root   4096 Aug  1 14:53 systemd-private-07cd6aacc5514427
812111a580dbcb67-systemd-resolved.service-GSXg4n
drwx-----  3 root      root   4096 Aug  1 14:53 systemd-private-07cd6aacc5514427
812111a580dbcb67-systemd-timesyncd.service-OpwRy4
ansible@ip-172-31-27-56:~$
```

```
ansible all -i /home/ansible/myinv -b -m file -a "path=/opt/devops
state=directory"
```

UC: You have a fleet of 100 servers and you want to run a command apt-get update in all 100 servers

command module

```
ansible -m command -a "apt-get update" webserver
```

How to add user manually?

```
adduser <username>
```

```
cat /etc/passwd // user will be created
```

```
su <username>
```

How to create user using ansible?

```
ansible -m user -a "name= neeraj password=password generate_ssh_key=yes" webserver
```

Check in target

```
cat /etc/passwd
```

```
su neeraj
```

```
cd /.ssh
```

```
ls
```

```
id_rsa id_rsa.pub
```

Yaml

```
yaml / yml -- yaml ain't markup language -- data serialization language -- representing data in a structured format
```

```
one basic rule to write any yaml
  indentation -- spaces -- NO TABS in a yaml
  key: value
```

scalars: key: value

```
3 data structures
scalars
```

```
name: venu
a: 5
b: 6.4
```

```
I
```

dictionary/map: holds multiple properties of an object

dictionary / map

```
car:
  name: bmw
  color: red
  model: sedan
  engine: 3L
```

Array/list: collection of multiple objects

list / array

```
- bmw
- audi
- KIA
- SKODA
```

```
cars:
  - bmw:
      color: white
      model: sedan
      engine:
        - 2L
        - 3L
        - 3.5L
  - audi
  - KIA
  - SKODA
```

ansible -- playbooks (yaml)

docker -- compose files (yaml)

kubernetes -- manifest files (yaml)

Playbook

```
task: a change that we want to implement on remote server -- copy a file / restart a service

we use pre defined ansible modules to write ansible tasks

play: set/list/series of tasks that are executed on set / list of remote server

playbook: list of plays
=====
=====

- name: play1
  hosts: web
  tasks:
    - name: task1
      file:  ## using file module we can create files & directories
      path: /tmp/devops
      state: directory  ## touch will create a file
    - name: task2
      copy:
        src: /tmp/hello.txt  ## by default source is always your controller machine
        dest: /tmp

- name: play1
  hosts: app
  tasks:
    - name: task1
      file:  ## using file module we can create files & directories
      path: /tmp/devops1
      state: directory  ## touch will create a file
    - name: task2
      copy:
        src: /tmp/hello1.txt  ## by default source is always your controller machine
        dest: /tmp
```

How to run playbook

```
~ ~ ~
- hosts: all
  become: yes
  tasks:
#### execute shell script & print the output
  - name : execute a shell
    script: /tmp/test.sh
    register: abc
  - debug:
    msg: "{{ abc.stdout_lines }}"
#### create a files
  - name: restart a service
    shell: echo "hostname"
    register: xyz
  - debug:
    msg: "value of xyz is: {{ xyz.stdout_lines }}"
~
~
~
```

script module execute script

step1: **nano /tmp/test.sh**

```
#!/bin/bash
echo "hi there, `hostname`"|
~
~
~
~
```

chmod 755 /tmp/test.sh

register module -> stores the output of the previous script module

debug: display the contents of the variable “The value of variable: {{{}}”

ansible-playbook <name of playbook. yaml> -i < inventory path>

It is serial execution, if one task fails it not starts another task

```
ansible@ip-172-31-30-77:~$ vi playbook.yml
ansible@ip-172-31-30-77:~$
ansible@ip-172-31-30-77:~$
ansible@ip-172-31-30-77:~$ ansible-playbook playbook.yml -i /home/ansible/myinv
PLAY [web] *****
TASK [Gathering Facts] *****
ok: [172.31.27.56]
```

What are Facts?

Cached all the information about the target hosts, before performing any task on the target host ansible first tries to gather all the information about the target host (performance impact). During the execution keeps in cached memory once execution completes, it frees the cached memory.

Whenever ansible try to implement any change on the remote server, it will try to gather some information about the target server, that is called the Facts.

Conditions?

when: (ansible_distribution get from the gather facts)

```

- hosts: all
  become: yes
  tasks:
    - name: install a webserver redhat/centos
      yum:
        name: httpd
        state: latest
        when: ansible_distribution == 'RedHat'
    - name: start the service
      service:
        name: httpd
        state: started
        when: ansible_distribution == 'RedHat'
    - name: install a webserver on ubuntu
      apt:
        name: apache2
        state: latest
        when: ansible_distribution == 'Ubuntu'
    - name: start the service
      service:
        name: apache2
        state: started
        when: ansible_distribution == 'Ubuntu'

```

UC: we want to switch to ubuntu user not to the root user?

become_user: ubuntu

```

- name: play1
  become: yes
  become_user: ubuntu
  hosts: web
  tasks:
    - name: task1
      file:
        path: /tmp/devops
        state: directory
    - name: task2
      copy:
        src: /tmp/hello.txt
        dest: /tmp
- name: play2
  hosts: web
  tasks:
    - name: task1
      file:
        path: /tmp/devops1
        state: directory
    - name: task2
      copy:
        src: /tmp/hello1.txt
        dest: /tmp

```

How to define variables?

```
---
- hosts: web
  become: yes
  vars:
    dockerceversion: 5:18.09.9~3-0~ubuntu-bionic
    dockercecli version: 5:18.09.9~3-0~ubuntu-bionic
    containerdioversion: 1.2.6-3
  tasks:
    - name: Add Docker GPG key
      apt_key:
        url: https://download.docker.com/linux/ubuntu/gpg
    - name: Install basic list of packages
      apt:
        name: ['apt-transport-https', 'ca-certificates', 'curl', 'gnupg2', 'software-properties-common']
        state: present
        update_cache: yes
    - name: Add Docker APT repository
      apt_repository:
        repo: deb [arch=amd64] https://download.docker.com/linux/{{ansible_distribution|lower}} {{ansible_distribution_release}} stable
    - name: Install Docker packages
      apt:
        name: "{{ item }}"
        name: ['docker-ce={{ dockerceversion }}', 'docker-ce-cli={{ dockercecli version }}', 'containerdio={{ containerdioversion }}']
        state: present
      with_items:
        - docker-ce={{ dockerceversion }}
```

Looping?

['apt-transport-https', 'ca-certificate', 'curl', 'gnupg2']

ignore_errors //module

Ansible Roles: split the bigger playbook in to smaller one

UC: reusability, breaking a playbook into multiple files. This simplifies writing complex playbooks, and it makes them easier to reuse.

Roles provide a framework for fully independent or interdependent collections of variables, tasks, files, templates and modules.

Roles are defined using the YAML files with a predefined directory structure.

Why Ansible Roles?

In ansible, the role is the primary mechanism for breaking a playbook into multiple files. This simplifies writing complex playbooks and it makes them easier to reuse.

Roles provide a framework for fully independent, or interdependent collections of variables, tasks, files, template and modules.

Roles are defined using yaml files with a predefined directory structure.

A role directory structure contains directories: defaults, vars, tasks, files, templates, meta, handler. Each directory must contain a main.yaml which contains relevant contents.

defaults: contains default variables for the role, variables in the default have the lowest priority, so they are easy to override.

vars: contains variables for the role, variables in vars have higher priority than variables in default directory

tasks: contains the main list of steps to be executed by the role

files: contains files which we want to be copied to the remote hosts. We don't need to specify a path of the resources stored in this directory.

templates: contains file template which supports modifications from the role. We use the Jinja2 templating language for creating templates

meta: contains metadata of role like an author, support, platforms, dependencies.

handlers: contains handlers which can be invoked by 'notify' directives and are associated with the service.

task is the mandatory folder

step1: create a top-level directory

```
mkdir roles
```

```
cd roles
```

step2: create an automation for role(tomcat)

```
mkdir tomcat
```

```
cd tomcat
```

step3: create a pre-defined folder structure

```
mkdir tasks // mandatory folder
```

```
cd tasks
```


step4: mandatory yaml (main.yaml)

```
touch main.yaml
```

Instead, or writing multiple tasks in main. yaml, write multiple .yaml files within the tasks folder, and call that file from the main.yaml (tasks folder).

```
touch install.yaml
```

```
touch uninstall.yaml
```

step5: how to execute this automation, as a single playbook? To call the entire automation called tomcat, we need to write a top-level file

step5.1: write a top-level file, resides one level above the roles

```
vi tomcat-role.yaml
```

```
- hosts: web
  become: yes
  roles:
    - tomcat  ## the name of the automation(folder name) under the roles directory
~
~
```

Instead of tasks, we need to call the role

step6: how to call remaining configuration (.yaml files)?

top level <tomcat-playbook.yaml> will **only** executes the main.yaml of the task folder then how to call other .yaml files?

```
- name: create a file
  file:
    path: /tmp/newtoday.txt
    state: touch

- name: config users
  include_tasks: config-user.yml

- name: install java
  include_tasks: install-java.yml
~
~
~
~
```

Using the include_tasks, here we face same problems all the tasks will be executed in sequence, means still we have a large playbook.

step7: condition

Solution: Using the conditional execution using the environment variable(action)

```

- name: create a file
  file:
    path: /tmp/newtoday.txt
    state: touch

- name: config users
  include_tasks: config-user.yml
  when: action == "install"

- name: install java
  include_tasks: install-java.yml
  when: action == "install"

- name: install tomcat
  include_tasks: install.yml
  when: action == "install"

- name: uninstall tomcat
  include_tasks: uninstall.yml
  when: action == "uninstall"
~
~

```

Without passing the action variable?

```

devops@controller:~$ ansible-playbook tomcat-role.yml -i myinv
PLAY [web] *****

TASK [Gathering Facts] *****
ok: [node2]
ok: [node1]

TASK [tomcat : create a file] *****
changed: [node1]
changed: [node2]

TASK [tomcat : config users] *****
fatal: [node2]: FAILED! => {"msg": "The conditional check 'action == \"install\"' failed. The error was: error while evaluating conditional (action == \"install\"): 'action' is undefined\n\nThe error appears to be in '/home/devops/roles/tomcat/tasks/main.yml': line 6, column 3, but may\nbe elsewhere in the file depending on the exact syntax problem.\n\nThe offending line appears to be:\n\n- name: config users\n  ^ here\n"}
fatal: [node1]: FAILED! => {"msg": "The conditional check 'action == \"install\"' failed. The error was: error while evaluating conditional (action == \"install\"): 'action' is undefined\n\nThe error appears to be in '/home/devops/roles/tomcat/tasks/main.yml': line 6, column 3, but may\nbe elsewhere in the file depending on the exact syntax problem.\n\nThe offending line appears to be:\n\n- name: config users\n  ^ here\n"}
PLAY RECAP *****

```

step7.1: How to pass the action (condition check action)

How to pass the action (name can be any either action or trigger) variable? Action variable pass from the command line

```

devops@controller:~$ ansible-playbook tomcat-role.yml -i myinv --extra-vars "action=uninstall"
[WARNING]: Found variable using reserved name: action

PLAY [web] *****

TASK [Gathering Facts] *****
ok: [node2]
ok: [node1]

TASK [tomcat : create a file] *****
changed: [node1]
changed: [node2]

TASK [tomcat : config users] *****
ok: [node2]
ok: [node1]

TASK [tomcat : install java] *****
ok: [node2]
ok: [node1]

TASK [tomcat : install tomcat] *****
ok: [node2]
ok: [node1]

TASK [tomcat : uninstall tomcat] *****
ok: [node2]
ok: [node1]

PLAY RECAP *****

```

```

devops@controller:~$ tree roles/tomcat/
roles/tomcat/
├── tasks
│   ├── config-user.yml
│   ├── install-java.yml
│   ├── install.yml
│   ├── main.yml
│   ├── uninstall.yml
│   └── upgrade.yml

```

step8: default and vars

nano defaults/main.yml // define your variables

```

a: 14
b: 5.7
~
~

```

nano vars/main.yml // define your variables

```

user: tomcat
group: tomcat
a: 45
~
~

```

vars have higher precedence over the default

step8.1: How to call these variable? (Within any .yaml e.g., main.yaml)

```
- name: var from defaults folder
  debug:
    var: a

- name: var from vars folder
  debug:
    var: user
```

```
devops@controller:~$ tree roles/tomcat/
roles/tomcat/
├── defaults
│   └── main.yml
├── files
│   ├── apache-tomcat-8.5.53.tar.gz
│   └── sampleapp.war
├── tasks
│   ├── config-user.yml
│   ├── install-java.yml
│   ├── install.yml
│   ├── main.yml
│   ├── uninstall.yml
│   └── upgrade.yml
└── vars
    └── main.yml
```

command line will take highest precedence.

step8: files

files: copy the files which you want to copy in target server

```
devops@controller:~/roles/tomcat$ mv /home/devops/sampleapp.war files/
devops@controller:~/roles/tomcat$ ls -l
total 16
drwxrwxr-x 2 devops devops 4096 Jan 23 16:02 defaults
drwxrwxr-x 2 devops devops 4096 Jan 23 16:03 files
drwxrwxr-x 2 devops devops 4096 Jan 23 15:52 tasks
drwxrwxr-x 2 devops devops 4096 Jan 23 16:02 vars
devops@controller:~/roles/tomcat$ cd files/
devops@controller:~/roles/tomcat/files$ ls -l
total 26212
-rw-rw-r-- 1 devops devops 10300600 Jan 23 14:17 apache-tomcat-8.5.53.tar.gz
-rw-rw-r-- 1 devops devops 16536021 Jan 23 14:18 sampleapp.war
```

```
- name: copy Tomcat tar file
  # get_url: url http://mirrors.estointernet.in/apache/tomcat/tomcat-8/48.5.53/bin/apache-tomcat-8.5.53.tar.gzdest=/opt/apache-tomcat-8.5.53 ta
r.gz
  copy:
    src: apache-tomcat-8.5.53.tar.gz
    dest: /tmp
```

structure:

```
devops@controller:~$ tree roles/tomcat/
roles/tomcat/
├── defaults
│   └── main.yml
├── files
│   ├── apache-tomcat-8.5.53.tar.gz
│   └── sampleapp.war
├── tasks
│   ├── config-user.yml
│   ├── install-java.yml
│   ├── install.yml
│   ├── main.yml
│   ├── uninstall.yml
│   └── upgrade.yml
└── vars
    └── main.yml
```

step9: templates (blue print)- host specific data (Jinjer templating)

cd templates

vi app.props

```
hostname: {{ ansible_hostname }}
hostip: {{ ansible_default_ipv4.address }}
~
~
~
~
~
~
```

step9.1: How to call these templates? (Within any. yaml e.g., main. yaml)

```
- name: use of templates
  template:
    src: app.props
    dest: /tmp
~
~
```

From where we will get the variables?

/* before executing a command on the host first ansible gather the facts */

How to execute on the host

```
- name: use of templates
  template:
    src: app.props
    dest: /tmp
~
```

```
devops@controller:~/roles/tomcat/tasks$ cd
devops@controller:~$ ansible-playbook tomatcat-role.yml -i myinv -e action=install
[WARNING]: Found variable using reserved name: action

PLAY [web] *****

TASK [Gathering Facts] *****
ok: [node2]
ok: [node1]

TASK [tomcat : create a file] *****
changed: [node2]
changed: [node1]

TASK [addcat : config users] *****
included: /home/devops/roles/tomcat/tasks/config user.yml for node1, node2
```

Validate

In node1

```
root@node1:/opt# cd /tmp
root@node1:/tmp# ls -l
total 10084
-rw-r--r-- 1 root root 10300600 Jan 23 16:14 apache-tomcat-8.5.53.tar.gz
-rw-r--r-- 1 root root 36 Jan 23 16:24 app.props
drwxr-x-- 2 tomcat tomcat 4096 Jan 23 16:15 hserfdata_tomcat
-rw-r--r-- 1 root root 0 Jan 23 16:23 newtoday.txt
drwx----- 2 devops devops 4096 Jan 23 14:06 ssh-qV2IPPyzum
drwx----- 3 root root 4096 Jan 23 14:03 systemd-private-dff3ba950ba343b1bcf1823d935c5fe2-apache2.service-szSrgG
drwx----- 3 root root 4096 Jan 23 14:03 systemd-private-dff3ba950ba343b1bcf1823d935c5fe2-chrony.service-MLNGGn
drwx----- 3 root root 4096 Jan 23 14:03 systemd-private-dff3ba950ba343b1bcf1823d935c5fe2-systemd-resolved.service-Rgu31a
root@node1:/tmp# cat app.props
hostname: node1
hostip: 10.128.0.11
root@node1:/tmp#
```

In node2

```
root@node2:/tmp#  
root@node2:/tmp# cat app.props  
hostname: node2  
hostip: 10.128.0.12  
root@node2:/tmp#
```

step10: handler (special tasks) which called through the notify

notify

purpose: repetitive/dependent tasks

nano /handler/main.yaml

```
- name: restart tomcat  
  service:  
    name: tomcat@  
    state: restarted  
~  
~  
~  
~  
~
```

step10.1: How to call

```
- name: Configure Tomcat users  
  template:  
    src: tomcat-users.xml  
    dest: /opt/tomcat/conf/  
  notify: restart tomcat
```

notify: name of the handler task

```
devops@controller:~$ tree roles/tomcat/  
roles/tomcat/  
├── defaults  
│   └── main.yml  
├── files  
│   ├── apache-tomcat-8.5.53.tar.gz  
│   └── sampleapp.war  
├── handlers  
│   └── main.yml  
├── tasks  
│   ├── config-user.yml  
│   ├── install-java.yml  
│   ├── install.yml  
│   ├── main.yml  
│   ├── uninstall.yml  
│   └── upgrade.yml  
├── templates  
│   └── app.props  
└── vars  
    └── main.yml
```

How to generate role structure automatically

ansible galaxy

cd /roles

ansible-galaxy init <automation name/docker>

ansible-galaxy init docker

```
devops@controller:~/roles$ ansible-galaxy init docker
- Role docker was created successfully
devops@controller:~/roles$ ls -l
total 8
drwxrwxr-x 10 devops devops 4096 Jan 23 16:46 docker
drwxrwxr-x  8 devops devops 4096 Jan 23 16:26 tomcat
devops@controller:~/roles$ tree docker/
docker/
├── README.md
├── defaults
│   └── main.yml
├── files
├── handlers
│   └── main.yml
├── meta
│   └── main.yml
├── tasks
│   └── main.yml
├── templates
├── tests
│   ├── inventory
│   └── test.yml
└── vars
    └── main.yml
```

ansible-galaxy has a repository, (galaxy.ansible.com), can share our automation

ansible-galaxy install <role directory>

galaxy.ansible.com # just like docker hub

What is Ansible Vault?

Encrypt the ansible playbook (AES250)

Ansible allows keeping sensitive data such as password or keys in encrypted files, rather than in plaintext in your playbook.

UC1: Encrypt the playbook at time of creation.

step1: create a new encrypted playbook

`ansible-vault create <vault>.yaml # as we enter ask for password`

step2: edit the encrypted playbook

`ansible-encrypted edit <vault>.yaml`

step3: change the password

`ansible-vault rekey <vault>.yaml`

UC2: Encrypt the existing playbook.

step1: Encrypt the existing playbook

`ansible-vault encrypt <target>.yaml`

step2: decrypt the encrypted playbook

`ansible-vault decrypt <target>.yaml`